

# MACHINE LEARNING IN PHYSICS

## GRAPH AUTOENCODERS

### PART 2

HARRISON B. PROSPER

PHY6937 / PHY4936

# Outline

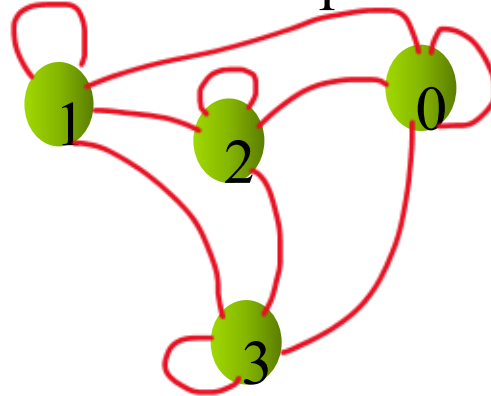
- Recap
- Graph Autoencoder – Part 2
- PyTorch Geometric
- Summary

Lecture based on notes by Dr. Claire David

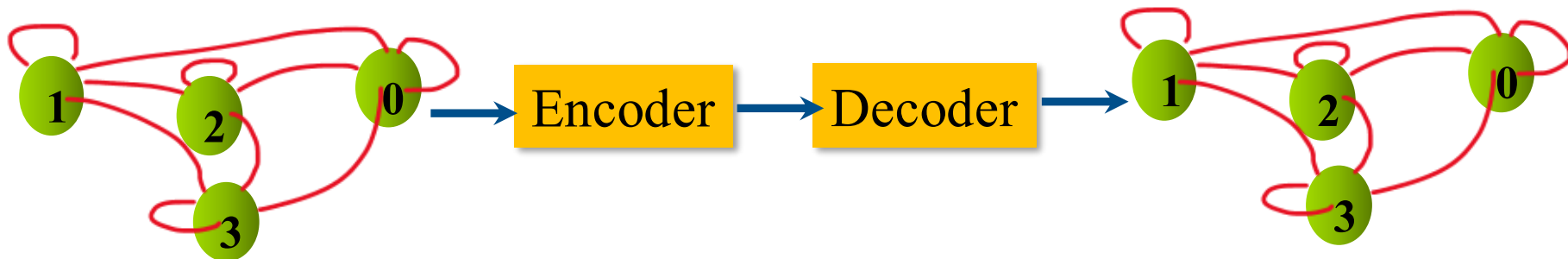
African Institute of Mathematical Sciences, Cape Town, South Africa.

# Recap (1)

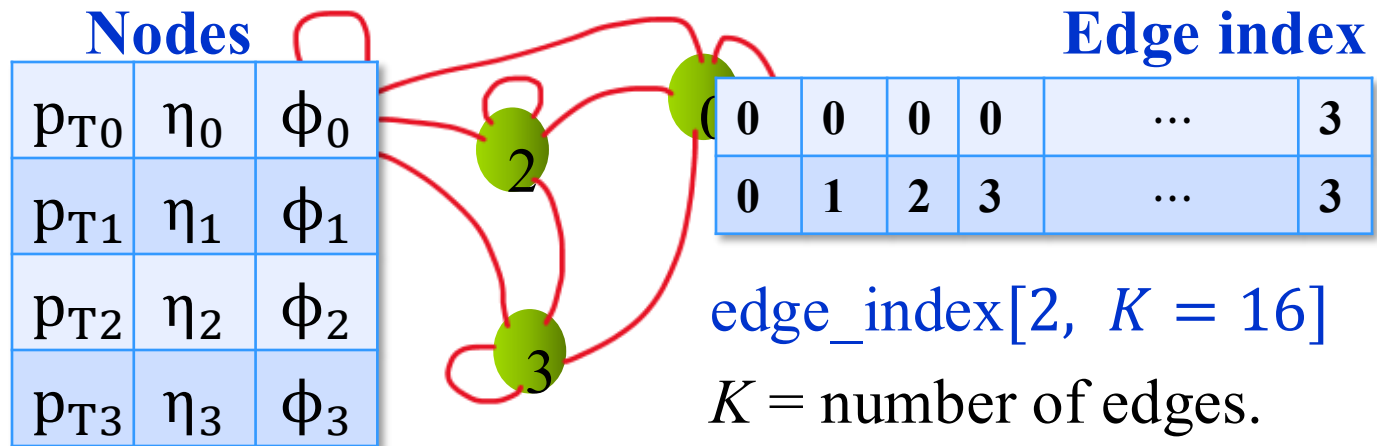
- A graph is a data structure that comprises nodes (or vertices) and edges:



- Our goal is to create a function (a GAE) that takes a graph as input, encodes it, and decodes it back to the same graph.



# Recap (2): Data Structures



$V[n = 4, d = 3]$

$\text{edge\_index}[2, K = 16]$

$K = \text{number of edges.}$

$d = \text{dimension of the input feature vectors.}$

target node index =  $\text{edge\_index}[0]$

source node index =  $\text{edge\_index}[1]$

edge features ( $\text{edge\_weights}[K]$ )

|             |             |             |             |     |             |
|-------------|-------------|-------------|-------------|-----|-------------|
| $e_{(0,0)}$ | $e_{(0,1)}$ | $e_{(0,2)}$ | $e_{(0,3)}$ | ... | $e_{(3,3)}$ |
|-------------|-------------|-------------|-------------|-----|-------------|

# GRAPH AUTOENCODER PART 2

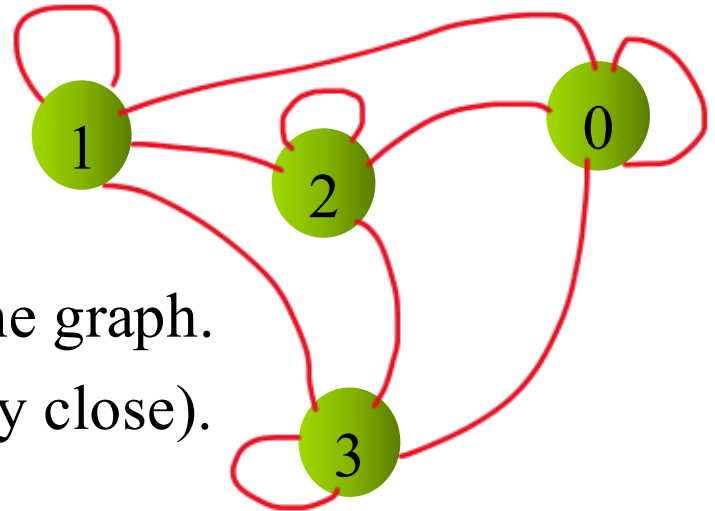
# Graph Autoencoder (1)

Our GAE computations so far are:

- $A = f(V)$  node adjacency (edge weights)
- $X = \sigma(VW_1^T + B_1)$  node embedding
- $X' = AX$  **node attention**

(Edges are defined by  $E = (\text{edge\_index}, \text{edge\_weights})$ )

$X' = AX$  is a simple example of the concept of **attention**: each target node “pays attention” to specific nodes in the graph. (in our example, those that are spatially close).



# Graph Autoencoder (2)

In addition to the computations,

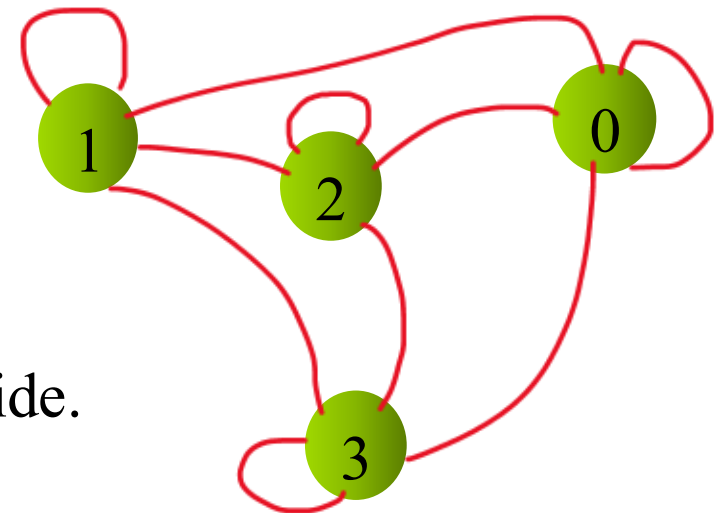
- $A = f(V)$  node adjacency (edge weights)
- $X = \sigma(VW_1^T + B_1)$  node embedding
- $X' = AX$  node attention

we'll perform another linear transformation:

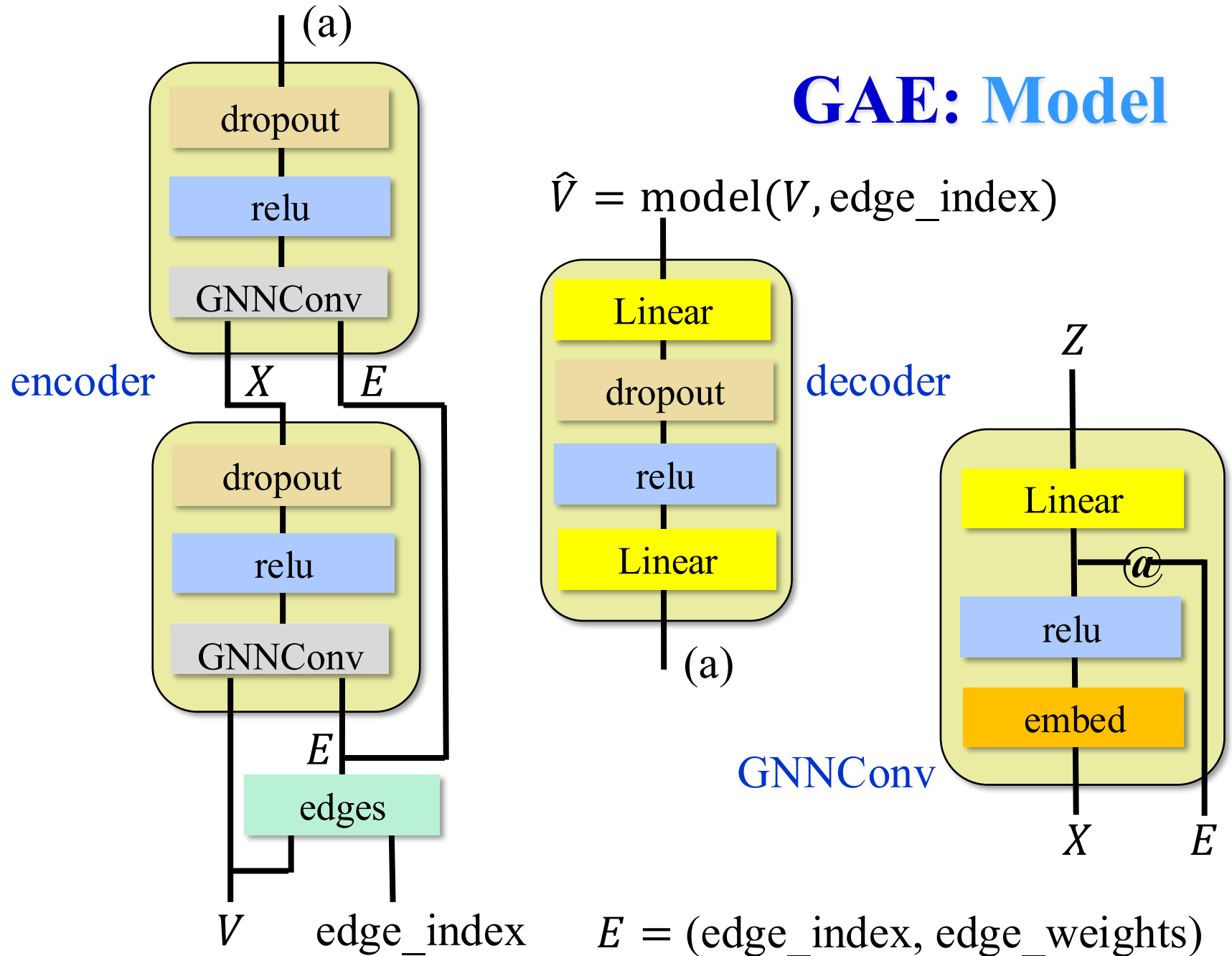
- $Z = X'W_2^T + B_2$

The matrices  $W_1$ ,  $B_1$ ,  $W_2$  and  $B_2$  are the parameters of our graph convolution operation, **GNNConv**.

The full model is shown on the next slide.



# GAE: Model





# PYTORCH GEOMETRIC (PYG)

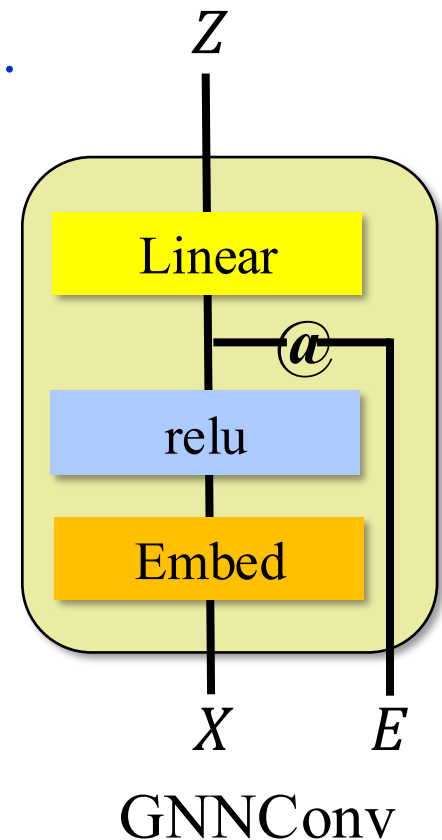
# PyTorch Geometric (1)

The toolkit **PyTorch Geometric (PyG)** solves two problems (among many others):

- How to do matrix algebra with **sparse matrices**.
- How to do **batch processing** of graphs.

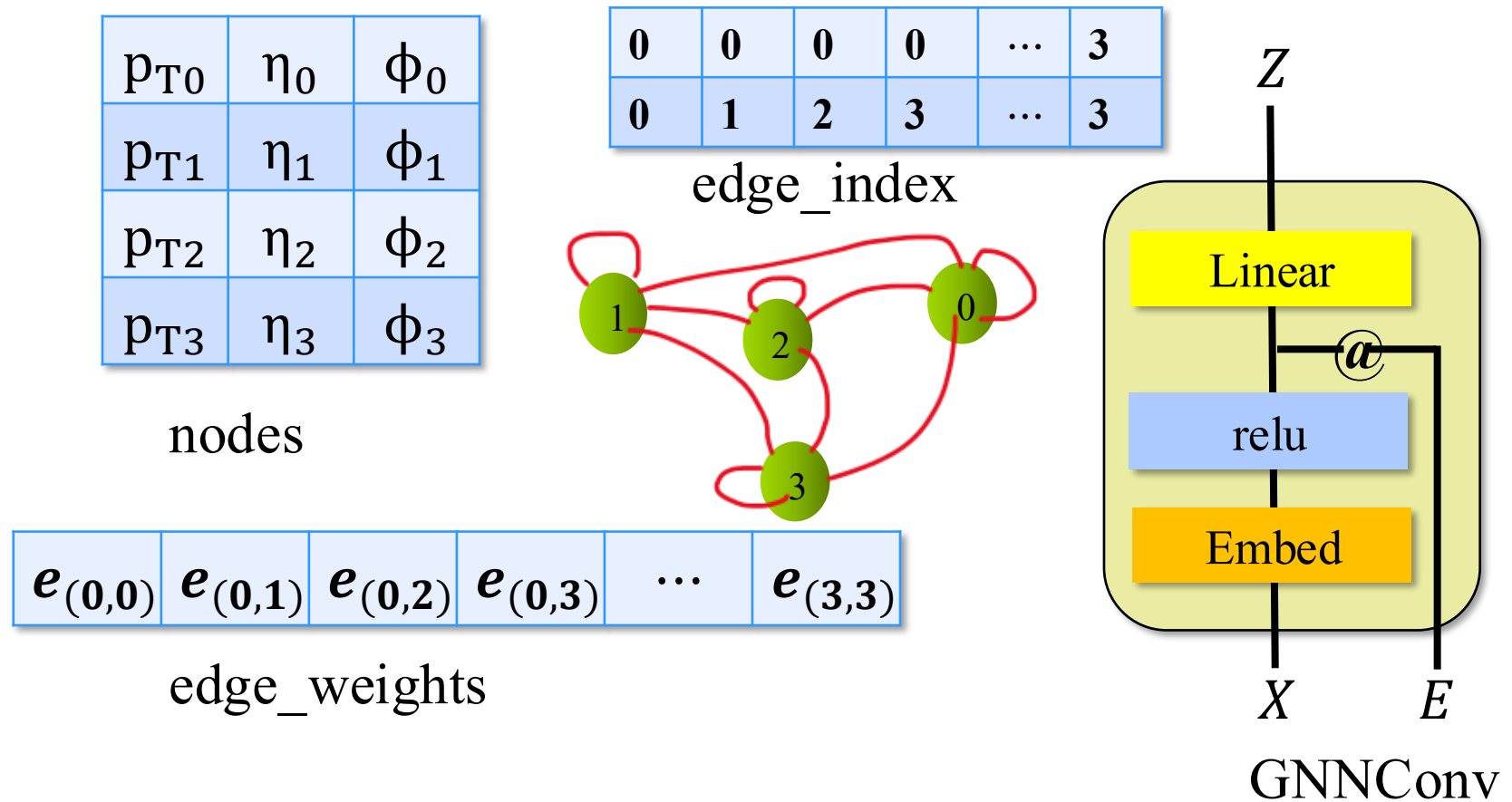
In our earlier GNN example, step **2** below used `torch.matmul( $A$ ,  $X$ )`. But in PyG (pi-gee)  $X' = AX$  is computed using  $X$  and the **edges**.

- 1.**  $X = \sigma(VW_1^T + B_1)$       node embedding
- 2.**  $X' = AX$       node attention
- 3.**  $Z = X'W_2^T + B_2$



# PyTorch Geometric (2)

In our 4-node graph example, PyG uses 3 data structures:

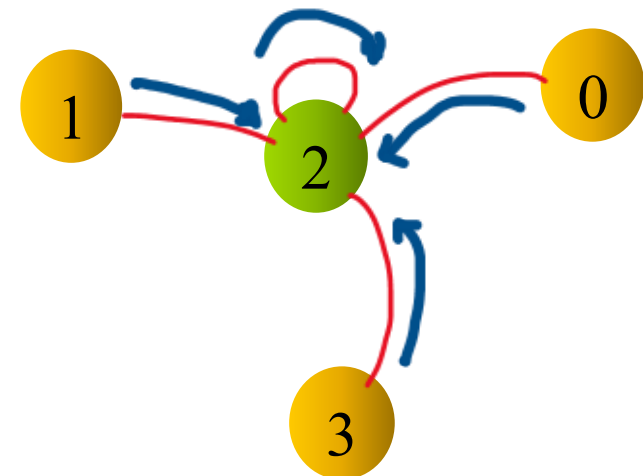
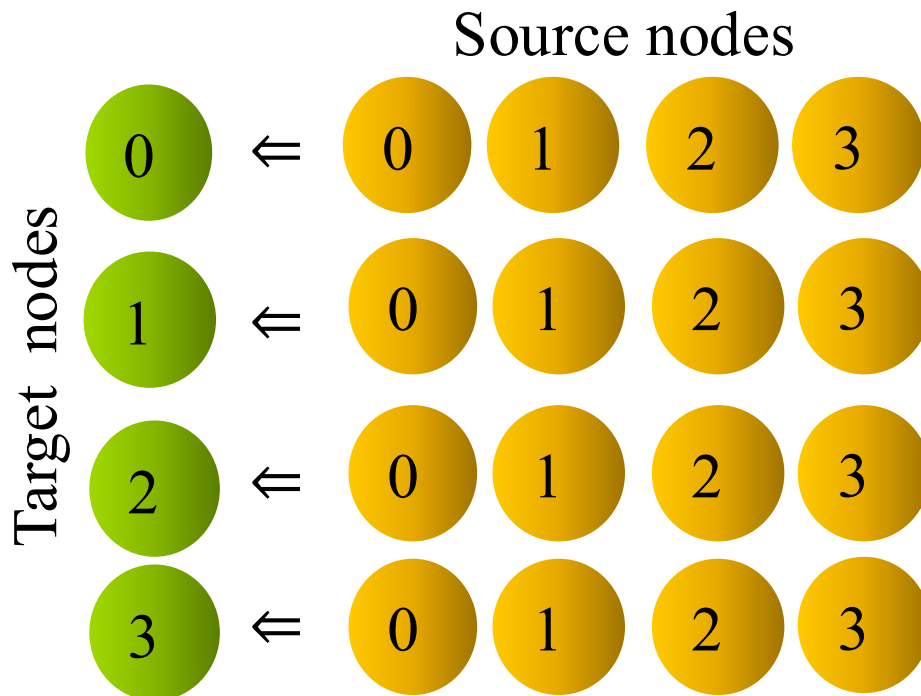


# PyTorch Geometric (3)

PyG solves the two problems by **serializing** graphs using these data structures. Each node is viewed as an *independent* entity linked to source nodes via the **edge\_index** and the associated **edge\_weight**.

|   |   |   |   |     |   |
|---|---|---|---|-----|---|
| 0 | 0 | 0 | 0 | ... | 3 |
| 0 | 1 | 2 | 3 | ... | 3 |

|             |             |             |             |     |
|-------------|-------------|-------------|-------------|-----|
| $e_{(0,0)}$ | $e_{(0,1)}$ | $e_{(0,2)}$ | $e_{(0,3)}$ | ... |
|-------------|-------------|-------------|-------------|-----|



# PyTorch Geometric (4)

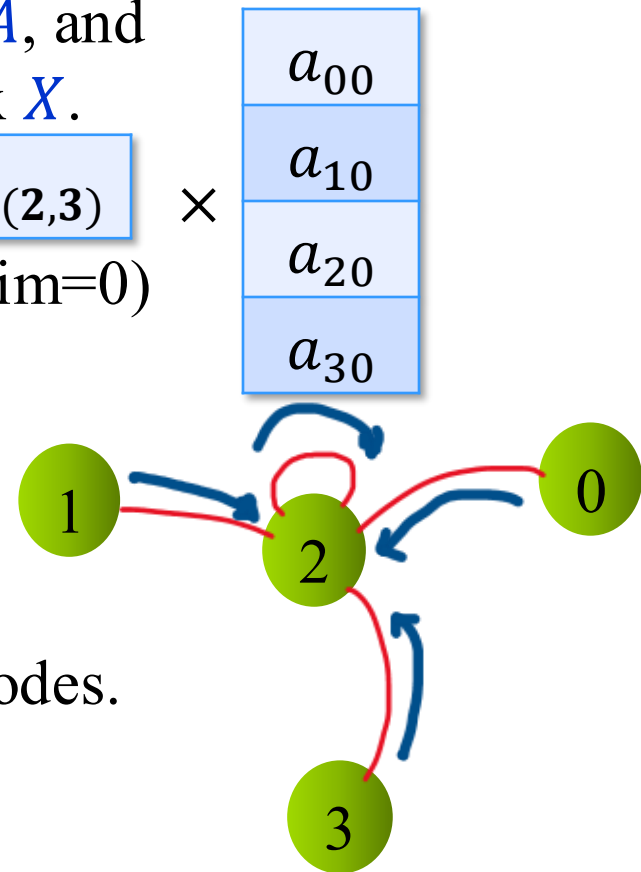
Consider the computation of  $X' = AX$ , where  $X$  is a tensor of embedded nodes of shape  $[n = 4, h]$ . Focus on target node **2**, which is associated with row **2** of matrix  $A$ , and consider the embedded feature **0** of matrix  $X$ .

$$\begin{bmatrix} e_{(2,0)} & e_{(2,1)} & e_{(2,2)} & e_{(2,3)} \end{bmatrix} \times \begin{bmatrix} a_{00} \\ a_{10} \\ a_{20} \\ a_{30} \end{bmatrix}$$

Before the *sum* in the *vertical* direction (dim=0) we collect the products into a column:

$$\begin{bmatrix} e_{(2,0)} a_{00} \\ e_{(2,1)} a_{10} \\ e_{(2,2)} a_{20} \\ e_{(2,3)} a_{30} \end{bmatrix}$$

Each row is a **message** associated with a single feature, namely, feature **0** from one of the 4 *source* nodes.



# PyTorch Geometric (5)

- This calculation is repeated with every other target node.
- Then the columns of products are stacked *vertically* to form a single column whose length equals  $K$  the number of edges in the graph.
- The procedure is repeated for each of the  $h$  features of the embedded nodes, and the columns are stacked *horizontally*.

|                   |     |                     |
|-------------------|-----|---------------------|
| $e_{(0,0)}a_{00}$ | ... | $e_{(0,0)}a_{0h-1}$ |
| $e_{(0,1)}a_{10}$ |     | $e_{(0,1)}a_{1h-1}$ |
| $e_{(0,2)}a_{20}$ |     | $e_{(0,2)}a_{2h-1}$ |
| $e_{(0,3)}a_{30}$ |     | $e_{(0,3)}a_{30}$   |
| $\vdots$          |     | $\vdots$            |
| $e_{(3,0)}a_{00}$ | ... | $e_{(3,0)}a_{0h-1}$ |
| $e_{(3,1)}a_{10}$ |     | $e_{(3,1)}a_{1h-1}$ |
| $e_{(3,2)}a_{20}$ |     | $e_{(3,2)}a_{2h-1}$ |
| $e_{(3,3)}a_{30}$ |     | $e_{(3,3)}a_{3h-1}$ |

# PyTorch Geometric (6)

- One thereby arrives at a tensor, comprising all **messages**, of shape  $[K, h]$ .
- In practice, this tensor is constructed by first expanding  $X$  into a tensor of shape  $[K, h]$  using  $X[\text{src\_nodes}]$ , where  $\text{src\_nodes} = \text{edge\_index}[1]$ . Then one does an elementwise product (that is, a **Hadamard product**) between  $\text{edge\_weights}$  and  $X[\text{src\_nodes}]$ .

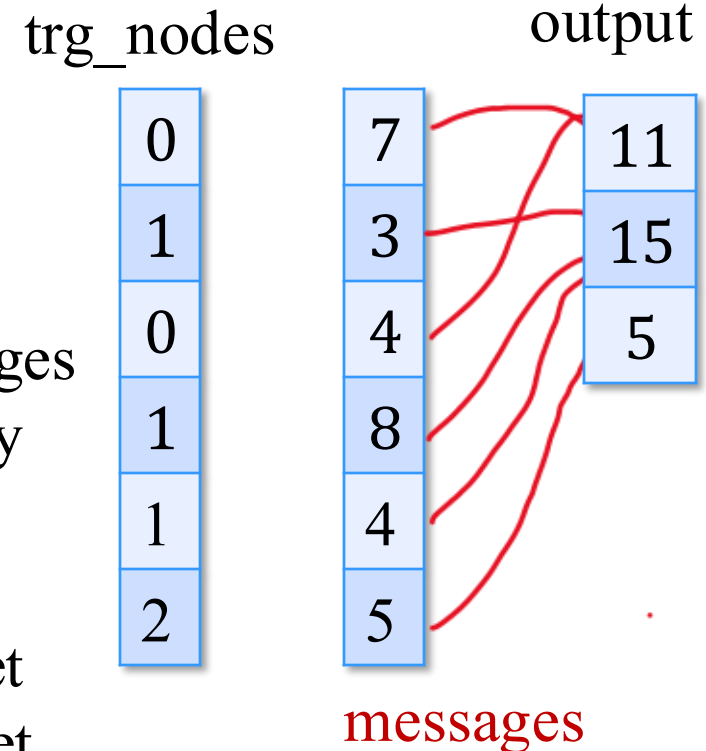
|                   |     |                     |
|-------------------|-----|---------------------|
| $e_{(0,0)}a_{00}$ |     | $e_{(0,0)}a_{0h-1}$ |
| $e_{(0,1)}a_{10}$ | ... | $e_{(0,1)}a_{1h-1}$ |
| $e_{(0,2)}a_{20}$ |     | $e_{(0,2)}a_{2h-1}$ |
| $e_{(0,3)}a_{30}$ |     | $e_{(0,3)}a_{30}$   |
| $\vdots$          |     | $\vdots$            |
| $e_{(3,0)}a_{00}$ |     | $e_{(3,0)}a_{0h-1}$ |
| $e_{(3,1)}a_{10}$ | ... | $e_{(3,1)}a_{1h-1}$ |
| $e_{(3,2)}a_{20}$ |     | $e_{(3,2)}a_{2h-1}$ |
| $e_{(3,3)}a_{30}$ |     | $e_{(3,3)}a_{3h-1}$ |

# PyTorch Geometric (7)

Finally, the PyG **scatter** function

```
output = scatter(messages,  
                  trg_nodes,  
                  dim=0,  
                  reduce='sum')
```

is used to perform the sum over messages *per* target node and *per* feature, thereby arriving at  $X' = AX$ , which is of shape  $[n, h]$ . In the graphic, the messages associated with a given feature to target node 0 are 7 and 4, while those to target node 1 are 3, 8, and 4.





# PyTorch Geometric (8): Sparsity

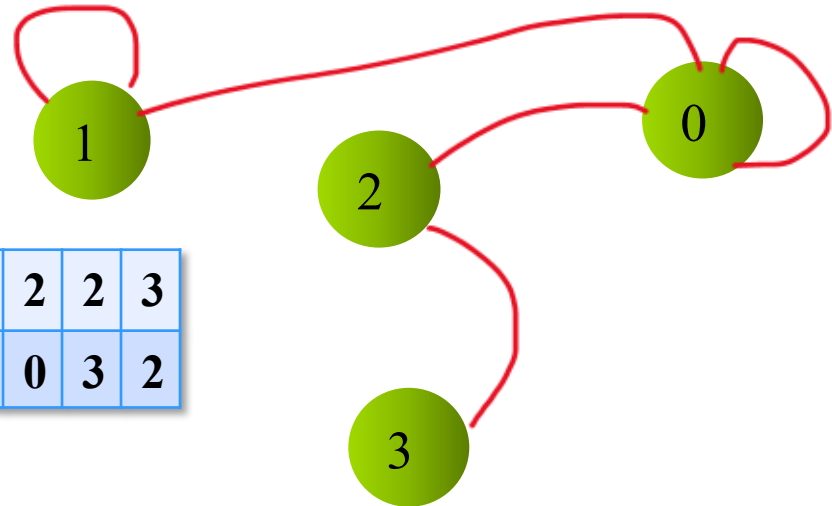
- The power of PyG arises from its ability to handle *sparse* graphs. Suppose our 4-node graph is connected as shown.

- The `edge_index` data structure of the graph is:

target nodes:

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 2 | 2 | 3 |
| 0 | 1 | 2 | 0 | 1 | 0 | 3 | 2 |

source nodes:



- One point we didn't emphasize but I hope was clear is that many of the links in our example graphs are **bi-directional**!
- The fact that our graph is now sparsely connected, changes nothing about how the matrix algebra is done!

# Summary

- A graph  $G = (X, E)$  comprises nodes  $X$  and edges  $E$ .
- A disparate array of datasets can be represented as graphs, including molecular structures, particle collision events, social networks, epidemics, etc.

- **PyTorch Geometric** is one of several toolkits that can be used to build and manipulate graphs and build and train sophisticated graph-based models.

